CodeCureAgent: Automatic Classification and Repair of Static Analysis Warnings

PASCAL JOOS, CISPA Helmholtz Center for Information Security, Germany
ISLEM BOUZENIA, CISPA Helmholtz Center for Information Security, Germany
MICHAEL PRADEL, CISPA Helmholtz Center for Information Security, Germany

Static analysis tools are widely used to detect bugs, vulnerabilities, and code smells. Traditionally, developers must resolve these warnings manually by analyzing the warning, deciding whether to fix or suppress it, and validating the correctness of the code change. Because this process is tedious, developers sometimes ignore warnings, leading to an accumulation of warnings and a degradation of code quality. Prior work suggests techniques to automatically repair static analysis warnings, but these are limited to specific analysis rules, cannot perform multi-file edits, and rely on weak validation mechanisms. This paper presents CodeCureAgent, an approach that harnesses LLM-based agents to automatically analyze, classify, and repair static analysis warnings. Unlike previous work, our method does not follow a predetermined algorithm. Instead, we adopt an agentic framework that iteratively invokes tools to gather additional information from the codebase (e.g., via code search) and edit the codebase to resolve the warning. CodeCureAgent detects and suppresses false positives, while fixing true positives when identified. We equip CodeCureAgent with a three-step heuristic to approve patches: (1) build the project, (2) verify that the warning disappears without introducing new warnings, and (3) run the test suite. We evaluate CodeCureAgent on a dataset of 1,000 SonarQube warnings found in 106 Java projects and covering 291 distinct rules. Our approach produces plausible fixes for 96.8% of the warnings, outperforming state-of-the-art baseline approaches by 29.2%–34.0% in plausible-fix rate. Manual inspection of 291 cases reveals a correct-fix rate of 86.3%, showing that CodeCureAgent can reliably repair static analysis warnings. The approach incurs LLM costs of about 2.9 cents (USD) and an end-to-end processing time of about four minutes per warning. We envision CodeCureAgent helping to clean existing codebases and being integrated into CI/CD pipelines to prevent the accumulation of static analysis warnings.

ACM Reference Format:
Pascal Joos, Islem Bouzenia, and Michael Pradel. 2026. CodeCureAgent: Automatic Classification and Repair of Static Analysis Warnings. 1, 1 (April 2026), 23 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Static analysis tools help developers identify potential bugs, vulnerabilities, and code smells [12, 30, 57], contributing to clean, reliable, secure, and maintainable software. Such tools check a code base for violations of a predefined set of rules, and report warnings when a rule is violated. Once a warning is reported, the task of resolving it traditionally falls on the developers.

Manually handling static analysis warnings poses several challenges. First, developers must understand the violated rule and why it is reported. Second, they must decide whether the warning is a true positive that warrants a fix, or a false positive that can be safely suppressed [19]. Third, developers must devise an appropriate fix and implement it, which may range from a quick, local edit

Authors' Contact Information: Pascal Joos, CISPA Helmholtz Center for Information Security, Germany, pascal.joos@outlook.de; Islem Bouzenia, CISPA Helmholtz Center for Information Security, Germany, bouzenia.islem@pm.me; Michael Pradel, CISPA Helmholtz Center for Information Security, Germany, michael@binaervarianz.de.

ACM XXXX-XXXX/2026/4-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

to substantially larger changes that span multiple files. Fourth, any new warnings introduced by the fix should undergo the same process. Finally, to be confident that the fix does not break functionality, the code change must be validated, e.g., by executing all relevant tests. This labor-intensive and repetitive workflow limits the practical adoption of static analyzers, causing unresolved warnings to accumulate in software projects. Moreover, introducing a static analyzer into an active project incurs substantial effort to address existing warnings, which can disrupt productivity and pollute build and testing pipelines with warning reports.

To assist developers in this process, automated classification and repair techniques have been proposed. Some studies focus on classifying warnings as true positives or false positives [28, 34, 52], while others aim to automatically fix warnings [11, 15, 21, 37]. In the literature, two families of approaches prevail: rule-based heuristics [2, 5, 15] and learning-based methods [7, 11, 13]. Rule-based approaches target a limited set of warning types and implement heuristic transformations for them, which is fast but covers only a few rules. A representative work in this category is Sorald [15], which implements rule-based fixes for 30 SonarQube rules (out of 479 default rules). Learning-based approaches have become increasingly popular, especially with the success of large language models (LLMs). For example, PyTy [10] trains a language model to repair static type errors in Python. Likewise, iSMELL [53] and CORE [49] query an LLM to automatically fix warnings.

Despite these advances, current approaches still suffer from several fundamental and practical limitations. To begin with, existing methods are confined to a single function or file scope, whereas some warnings require modifications across multiple files. Moreover, current repair techniques treat all warnings as true positives without verifying whether they should be fixed [15, 21, 28, 52]. This ignores the fact that static analyzers emit false positives, which developers would simply suppress [19], instead of devising a "fix" for a non-existent problem. Additionally, current approaches [16, 49] lack robust validation of the suggested fixes, as they are not subjected to systematic testing. Finally, LLM-based methods inherit the usual drawbacks of language models, namely outdated training data [4, 33] and hallucination [7, 18].

This paper presents CodeCureAgent, an end-to-end approach for classifying and fixing static analysis warnings. CodeCureAgent follows an agentic paradigm, leveraging an LLM in a fully autonomous loop, where an agent explores the project and makes edits via tools similar to those used by human developers. The agentic architecture allows CodeCureAgent to retrieve relevant documentation and learn from previous attempts, alleviating typical LLM limitations such as outdated knowledge or hallucination. The agent is divided into two sub-agents. The first sub-agent determines whether a warning is a true positive or a false positive, thereby avoiding the unfounded assumption that every warning requires a fix. Depending on this classification, the second sub-agent either repairs the warning (true positive) or suppresses it (false positive). Unlike prior work, CodeCureAgent can modify multiple files and multiple locations within a file, enabling it to address complex warnings more effectively. A key component of CodeCureAgent is the *change approver*, which builds the project, confirms that the target warning disappears without introducing new warnings, and runs the test suite to identify and reject low-quality patches. We evaluate CodeCureAgent on a dataset of 1,000 SonarQube warnings extracted from 106 Java projects. Running CodeCureAgent on this dataset produces plausible fixes for 968 warnings (96.8%), with an average processing time of four minutes per warning. Overall, CodeCureAgent classifies 69.6% of the warnings as true positives and 30.4% as false positives, highlighting the prevalence of false positives. Manual inspection of 291 warnings with distinct rules shows that 91.8% of the classifications are correct, 89.0% of the fixes are plausible, and 86.3% of the fixes are correct according to our manual validation. CodeCureAgent handles warnings of varying difficulty: Of the 968 plausible fixes, 424 modify multiple lines, and 27 affect multiple files. Under the same evaluation process, Sorald [16] can process 62/1,000 warnings and produces plausible fixes for 69.4% of them, whereas

```java
import lombok.Value;

public class PlayerEnchantOptions {
    // Annotation to auto-generate constructor, getters, setters
    @Value
    public class EnchantOptionData {
        private final String enchantName; // Raises SonarQube warning S1068:
                                          // "Unused 'private' fields should be removed"
    }

    public void addEnchantOption(String enchantName){
        this.options.add(
            // Call to auto-generated constructor
            new EnchantOptionData(..., enchantName, ...)
...
```

Listing 1. Simplified example of false positive reported in project Nukkit.

CodeCureAgent produces plausible fixes for all 62 (+30.6%). iSMELL [53] and CORE [49] yield plausible fixes for 62.8% and 67.6% of the 1,000 warnings, respectively, which is 34.0% and 29.2% less than CodeCureAgent.

Given its novel agentic design, an end-to-end pipeline (classification, fixing, and validation), and low operational cost, CodeCureAgent offers effective automatic repair of static analysis warnings. We anticipate that our results will encourage developers to adopt static analyzers without fearing excessive time or monetary overhead.

In summary, our contributions are:

- The first LLM agent for addressing static analysis warnings, which autonomously explores the codebase and applies edits via tools.
- A three-stage design that (i) classifies warnings as true positives or false positives, and (ii) either fixes or suppresses them, and (iii) validates the changes via building, re-analyzing, and testing.
- A comprehensive evaluation on a dataset of 1,000 warnings from 106 real-world projects, demonstrating that CodeCureAgent outperforms state-of-the-art baselines by a large margin.
- The code and data are publicly available at https://github.com/sola-st/CodeCureAgent.

2 Motivation and Running Example

Before presenting the details of CodeCureAgent, the following motivates our work by discussing key challenges in fixing static analysis warnings and illustrating them with real-world examples.

2.1 Challenge 1: Not All Warnings Need to be Fixed

Static analyzers are designed to identify issues in code. Unfortunately, not all warnings reported by static analysis are useful to developers, e.g., because some warnings are false positives or the developers do not consider them important enough to address [20, 23, 43]. Instead, developers often choose to suppress warnings, e.g., by adding annotations or comments to the code [19].

Listing 1 illustrates an example of a false positive, which when fixed in the way suggested by the analysis documentation, will break the code's functionality. The warning indicates that the `enchantName` field is private but never used in the scope of the class `EnchantOptionData`. The documentation associated with the analysis rule instructs the developer to remove the unused private field from the class. While this change seems reasonable at first glance, the class turns out to use an `@Value` annotation from a third-party library (Lombok) that automatically generates constructors, getters, and other boilerplate methods. Interestingly, a later version of the static analyzer (SonarQube v10.6) acknowledges this false positive and adds special treatment for it.

```java
public class ChangeInfo {
    public int _number; // Raises SonarQube warning S1104:
                        // "Class variable fields should not have public accessibility"
    ...
}
```

Listing 2. Running example. Simplified code with warning of rule S1104 in project gerrit-rest-java-client.

Prior work on automatically fixing static analysis warnings [11, 16, 49] ignores the possibility of false positives and assumes that all warnings need to be fixed. Instead, our approach addresses this challenge by first classifying whether a warning is a true positive or a false positive and then either fixing or suppressing the warning, respectively.

2.2 Challenge 2: Choosing the Best Fix for a Warning May Require Non-Local Context

Once a warning has been determined to be worth fixing, the next challenge is to find an appropriate fix. For many warnings, there are multiple possible fixes, and choosing the best one may require understanding the code context beyond the warnings location. To illustrate this challenge, consider our running example in Listing 2. The analysis warns about the public accessibility of the field `_number` in the class `ChangeInfo`. The documentation associated with the violated analysis rule describes two options for fixing the warning: (1) make the field private and provide getter and setter methods, or (2) mark the field as `public final`. Deciding which option is most appropriate requires understanding the usage of the field beyond the lines surrounding the warning: Option (1) is suitable if instances of the class may have different values for the field and the value may change, while option (2) is only applicable if the field is intended to never change its value.

One potential approach for addressing this challenge is to provide relevant code snippets to the LLM. However, it is not trivial to determine which parts of the code are relevant, especially when aiming to support a wide range of warning types. Instead of hard-coding heuristics for selecting relevant code snippets, CodeCureAgent addresses this challenge by equipping an LLM agent with tools to actively gather code and other information relevant to a warning, e.g., by searching for references to the field in other parts of the codebase or by looking up documentation.

2.3 Challenge 3: Fixes May Span Multiple Hunks and Multiple Files

While some warnings can be fixed by modifying a few lines of code around the warning location, other warnings require more extensive changes that may span multiple hunks and multiple files. This poses a challenge for automated repair approaches, especially those that operate on a single hunk [11] or single file [49]. For the running example in Listing 2, the appropriate fix is to make the `_number` field private and provide getter and setter methods. Because the field is accessed by other classes, the fix also requires modifying all references to the field outside of the class by replacing any reference to the `_number` field with a call to the getter or setter method. That is, the complete fix spans multiple hunks and multiple files.

Our approach addresses this challenge in two ways. First, we provide the LLM agent with a tool that can modify multiple locations in multiple files in a single step. Second, the agent may invoke this tool multiple times, thereby allowing it to iteratively refine its fix.

2.4 Challenge 4: Candidate Fixes May Break Functionality or Introduce New Warnings

A final challenge faced by a developer or an automated repair approach is to ensure that the proposed fix does not break the functionality of the code or introduce new static analysis warnings. For example, a naive fix for the running example in Listing 2 could be to make the field `_number` private and provide getter and setter methods with the names `get_number` and `set_number`. However,

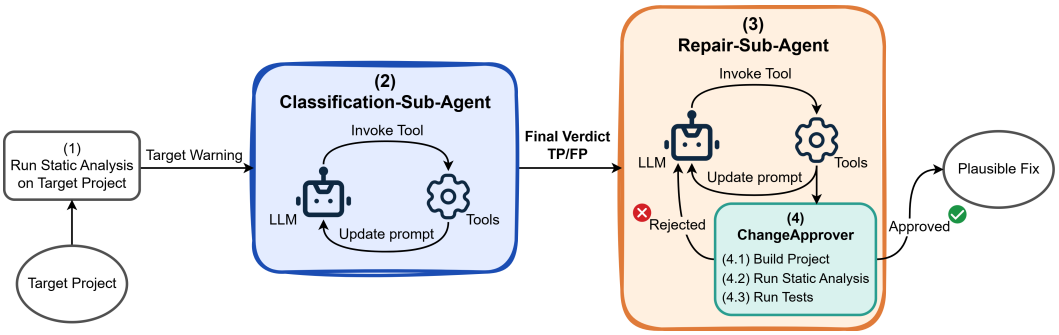


Fig. 1. Overview of CodeCureAgent.

this fix would introduce two new warnings of rule S100: "Method names should comply with a naming convention", as the method names do not follow the `camelCase` naming convention.

State-of-the-art approaches either do not validate their fixes beyond checking that the targeted warning disappears [11, 16] or rely on weak validation mechanisms, such as asking an LLM to rank candidate fixes [49]. Instead, CodeCureAgent addresses this challenge by employing a three-step validation process that (1) builds the project, (2) verifies that the target warning disappears without introducing new warnings, and (3) runs the test suite to check for regressions.

3 Approach

3.1 Overview

Figure 1 provides an overview of CodeCureAgent, which adopts an agentic approach to autonomously classify and fix static analysis warnings. The approach consists of a preparation stage, followed by two sub-agents: one that classifies a warning as a true positive (TP) or a false positive (FP), and another that fixes or suppresses the warning accordingly.

In the preparation stage (1), CodeCureAgent applies static analysis to the target project. The output is a list of warnings, which CodeCureAgent processes one at a time. For each warning, the *classification sub-agent* (2) determines whether the warning is a TP or FP. The agent operates in cycles, each consisting of: (i) constructing or updating a dynamic prompt, (ii) querying an LLM with the prompt, and (iii) executing a tool as specified by the LLM response. Equipped with a toolbox that enables the agent to explore the codebase and the documentation associated with warnings, it issues a classification once sufficient knowledge has been gathered or the computational budget is exhausted. Once the agent has produced the final verdict, control passes to the second sub-agent.

The *repair sub-agent* (3) edits the codebase to address the warning by either fixing it (in the case of a TP) or suppressing it (in the case of a FP). When the repair sub-agent proposes a patch, the *change approver* (4) validates the plausibility of the fix. This validation consists of three steps: confirming that the project builds without compilation errors, rerunning the static analyzer to check that the target warning disappears and that no new warnings are introduced, and executing the project's test suite to prevent regressions. If any check fails, the fix is rejected, and contextual feedback describing the failure is appended to the prompt, enabling the repair sub-agent to gather more information and propose an improved fix. Once the change approver accepts a fix, it is returned to the user.

The remainder of this section details each of the steps and components of CodeCureAgent.

Table 1. Input fields given to CodeCureAgent with illustrative examples.

Field	Description	Example
Repository	Target project at a specific commit	https://github.com/uwolfer/gerrit-rest-java-client at de49b4e
RuleKey	Key of the violated rule	java:S1104
FilePath	Path to the file with the warning	src/main/java/com/google/gerrit/extensions/common/ChangeInfo.java
StartLine	Line where the warning is raised	82
RuleName	A short description	Class variable fields should not have public accessibility
SpecificMessage	Context-specific warning message	Make `_number` a static final constant or non-public and provide accessors if needed.
RuleType	Type of the rule	Code smell

3.2 Running Static Analysis on Target Project

To prepare the input for running CodeCureAgent, we run a static analysis on the target project. Static analysis tools may have different output formats and objectives, but in general, they check the program against a set of rules and output a list of locations where these rules are violated. In our experiments, we use SonarQube as the static analyzer, but CodeCureAgent does not rely on any specific property of SonarQube and could be adapted to work with other static analyzers. We consider a warning to consist of seven fields, as listed in Table 1. The last column of the table provides examples of these fields for the warning in our running example (Listing 2).

3.3 Classification Sub-Agent: True Positive (TP) vs. False Positive (FP)

Given a target warning, CodeCureAgent first determines whether the warning is a TP or a FP. We place this classification stage before the actual repair because prior work shows that FPs introduce noise and require different handling than TPs [19, 28, 52]. We address this challenge with an agent-based classifier called the *classification sub-agent*. Consistent with general agent architectures [6, 51], the classification sub-agent operates in iterative *cycles*, starting by (i) constructing a prompt that includes the task description, the input warning, and the set of accessible tools, (ii) sending the prompt to an LLM, and (iii) parsing the LLM's response into a tool call and executing the tool. These three steps repeat until the classification agent has accumulated sufficient information. At the end, the agent invokes a dedicated tool to emit the final decision on whether the warning is a TP or FP. Below, we provide further details on prompt design, LLM interactions, and tool invocation.

3.3.1 LLM Prompting. Each cycle begins with the explicit construction of a prompt. The prompt is assembled by concatenating the six sections enumerated below, each of which supplies a distinct piece of information to guide the LLM's behavior.

(1) **Role and objectives.** This opening segment assigns the agent its role (classifier) and objective: to decide whether the supplied warning is a TP or a FP. By framing the task up front, the model knows exactly what outcome it must produce.
(2) **Constraints.** Here, we introduce constraints and prohibitions. For example, the agent is restricted from seeking user assistance and from introducing new warnings.
(3) **Available tools.** A concise list of the tools the agent may invoke, along with brief usage instructions (Table 2). Because the LLM must emit its response in a predefined tool-call format, this section acts as a lookup table that maps high-level intents (such as "read documentation") to concrete API calls.

Example Agent Trajectory (shortened)

Cycle 1
Tool Call: ReadDocumentation(rule_key="S1104")
Tool Output: Class variable fields should not have public accessibility...
—
Cycle 2
Tool Call: ReadLines(file_path="src/main/.../ChangeInfo.java", start_line=70, end_line=90)
Tool Output:

```
...(shortened)
public int _number;
...(shortened)
```

... (jump to c7)
Cycle 7
Tool Call: AnswerClassificationQuestions(answerQ1="Yes, the rule violation is correctly raised...)
Tool Output: Your answer to question Q1 has been recorded.
... (jump to c12)
Cycle 12
Tool Call: GiveFinalVerdict(verdict="True Positive", rationale="The field _number is public and accessed directly in multiple files ...")
Tool Output: — Transition to Repair Sub-Agent —

Listing 3. Shortened agent trajectory with tool calls for the motivating example (Listing 2).

(4) **Input warning.** This describes the warning under investigation, as illustrated in Table 1.
(5) **Agent history.** The prompt incorporates a chronologically ordered log of all previous cycles, recording the agent's thoughts, any tool calls made, and the corresponding tool responses. This history serves as the agent's memory, allowing the LLM to build upon earlier deductions rather than starting from scratch in each cycle.
(6) **Definition of response format.** Finally, we define the syntax to be used when issuing a tool call (a JSON-like output containing the tool name and arguments). By enforcing a strict format, downstream processing can reliably parse the output and execute the requested tool.

Given the prompt with these six sections, the LLM is called and its answer is expected to be a tool call in the specified format.

3.3.2 Tool Invocation. After the model produces a response, CodeCureAgent validates that the response adheres to the defined format. If parsing the response is not possible, the parsing error is appended as feedback to the agent history, helping the agent generate correct responses that can be mapped to a valid tool call in the future. If parsing is successful, the approach invokes the corresponding tool. Once a tool finishes execution, it returns its recorded output. CodeCureAgent then adds this output to the agent history, ensuring that subsequent iterations incorporate the result into the prompt. To avoid exceeding the model's context window, the output of a tool invocation is always truncated to the first 125K tokens. This precaution prevents multiple long outputs from exhausting the available context when added to the agent history. An example of tool invocations by the agent over multiple cycles is given in Listing 3. The set of tools enabled for the classification sub-agent is given in Table 2.

3.3.3 Classification. Static analyzers are known to generate large numbers of warnings, many of which are irrelevant or difficult to act upon. This makes classification a crucial first step: distinguishing TPs from FPs prevents wasted repair attempts and avoids introducing incorrect changes [28, 52].

Once the classification agent has collected sufficient information, it can call the tool *Answer classification questions* to address three guiding questions that support the decision on warning classification. In brief, these questions assess: (i) whether the warning is correctly raised and the rule description applies in this specific case, (ii) whether the developer may have intentionally

Table 2. Tools available in different parts of CodeCureAgent.

		Sub-Agents		
Tool name	**Tool description**	**C-SA**	**R-SA Fix**	**R-SA Supp**
Collecting information				
Read documentation	Retrieves documentation for a given analysis rule.	✓	✓	
Read lines	Reads a range of lines in a given file.	✓	✓	✓
Find references	Finds all project-local references, such as call sites or usages of a symbol.	✓	✓	
Find definition	Retrieves the definition of a project-local symbol, such as methods and classes.	✓	✓	
Search patterns	Searches for given patterns.	✓	✓	
Code Editing				
Write fix	Used to propose a fix or suppression.		✓	✓
Control tools				
Formulate plan	Creates/updates warning patching plan.		✓	
Answer classification questions	Gives an answer to the three classification questions.	✓		
Give final verdict	Gives a classification verdict (TP or FP).	✓		
Goals accomplished	Declare end of task with success; allowed only when checks pass.		✓	✓

C-SA = Classification Sub-Agent **R-SA Fix** = Repair Sub-Agent (Fixing a Warning)
R-SA Supp = Repair Sub-Agent (Suppressing a Warning)

violated the rule (e.g., for functional or design reasons), and (iii) whether the warning can be fixed without breaking existing functionality. The agent gives a yes/no answer and an explanation that refers to collected context to avoid baseless or hallucinated answers.

Following that, the agent issues a final verdict by calling the tool *Give Final Verdict* which asks the agent for the final classification decision and its rationale. Depending on the classification result, CodeCureAgent then proceeds to the next stage: if the warning is a TP, the repair sub-agent attempts to fix it, otherwise it attempts to suppress it.

3.4 Repair Sub-Agent: Fixing or Suppressing the Warning

The repair sub-agent takes as input the target warning and the final classification decision made by the classification sub-agent. Based on the classification, the sub-agent either fixes the warning or suppresses it. It follows the same agentic approach as the classification sub-agent, but with a different prompt and a tailored set of tools (see Table 2).

3.4.1 Fixing True Positives. If the warning is classified as a TP, the repair sub-agent attempts to fix it by suggesting modifications to the code. For this purpose, the agent has access to the same information-gathering tools as the classification sub-agent. Using these tools, the agent can retrieve additional code context, examine relevant definitions or references, and identify lines requiring modification. Tools such as *Find references* and *Find definition* are particularly helpful when changes may affect multiple files. An additional tool specific to the repair task is *Formulate Plan*. This tool allows the agent to draft a plan for how it intends to fix the warning, which is then added to a dedicated section of the prompt. The plan can be updated iteratively, allowing the agent to adapt to previous attempts' failures.

```
[{ "file_name": "src/main/java/.../ChangeInfo.java",
    "insertions": [
    { "line_number": 16,"new_lines": ["import com.google.gson.annotations.SerializedName;"] },
    { "line_number": 104,"new_lines": [
        "@SerializedName(\"_number\")",
        "private int number;",
        "public int getNumber() { return number; }",
        "public void setNumber(int n) { this.number = n; }"
        ]}
    ],
    "deletions": [82]
},
{ "file_name": "src/main/java/.../ChangesRestClient.java",
    "insertions": [
    { "line_number": 157,"new_lines": ["return id(info.getNumber());"] }
    ],
    "deletions": [157]
},
{ "file_name": "src/test/java/.../RealServerTest.java",
    "insertions": [
    { "line_number": 75,"new_lines": ["int changeNum = changeInfoList.get(0).getNumber();"] },
    { "line_number": 78,"new_lines": ["Truth.assertThat(change.getNumber()).isEqualTo(changeNum);"] }
    ],
    "deletions": [75, 78]
}]
```

Listing 4. Shortened fix of motivating example showing insertions, deletions, and multiple files.

Once the agent has gathered sufficient information, it calls the *Write fix* tool to propose a modification, expressed in a specified JSON format. Listing 4 shows an example fix for the warning introduced in Listing 2. Our running example also illustrates that repairing some warnings may require coordinated changes across multiple files. To do so, insertions and deletions are expressed through the `insertions` and `deletions` fields for each file. Once submitted, the proposed fix is applied to the target project and validated using the change approval mechanism described in Section 3.5. If the agent approaches the end of its cycle budget (i.e., five or fewer cycles remaining), a special message is added to the prompt to strongly urge the agent to produce a fix.

3.4.2 Suppressing False Positives. If the warning is classified as a FP, the repair sub-agent is tasked with suppressing it. Since this is more straightforward than fixing, the sub-agent is restricted to a smaller toolset: *Read lines*, *Write fix*, and *Goals accomplished*. Most static analyzers provide specific mechanisms for suppressing warnings [19]. In our experiments, we use SonarQube, which offers two ways to suppress Java warnings. The first is to add an inline comment `//NOSONAR` to the line where the warning is raised, which suppresses all warnings on that line. The second is to use the `@SuppressWarnings({"java:S..."})` annotation on a language construct, such as a method or class, which suppresses all warnings of the specified rule within that scope. We prompt the LLM to prefer the first option, as the second may inadvertently suppress multiple warnings if applied to large scopes, such as classes, and may even suppress warnings that could appear in future code additions [19]. To guide the agent in suppressing warnings, the prompt also includes the explanation given by the classification sub-agent as part of its final verdict.

3.5 Approving Changes

After applying the fix generated by the repair sub-agent, the change approver validates the fix. If the fix is approved, the approach marks it as *plausible* and reports it to the user. Otherwise, the repair sub-agent receives a rejection message with details. The change approver evaluates a proposed fix through three validation steps described below.

3.5.1 Building the Project. In the first step, the change approver attempts to build the modified project. If the build fails, the fix is rejected, and the compilation errors are provided as feedback to the agent. Because build outputs are often lengthy and may contain information from all build substeps, the approach extracts only the relevant compilation errors and ignores standard output. For each error, it reports the file path, line number, the code line that triggered the error, and a diff-like view between the original code and the failing fix.

3.5.2 Running the Static Analysis. If the build succeeds, the change approver further validates the fix by running the static analysis on the modified codebase. This step is inspired by techniques that learn patches from historical fixes to satisfy static analyzer rules [2, 5]. Validation is successful if the target warning has been removed from the analysis report and no new warnings have been introduced. Otherwise, the fix is rejected.

The location of a warning may change after applying a fix due to inserted or deleted lines. To accurately determine whether the target warning has been removed and whether new warnings have been introduced, our approach tracks all line changes in modified files and maps warnings between the original and modified lines. Using this mapping, the approach checks whether the target warning is still present in the corresponding lines after applying the fix. If not, it confirms that the warning has been successfully removed. Similarly, the approach detects newly introduced warnings by mapping warnings from the updated report back to the original lines. If no corresponding entry exists, the approach considers the warning as new. The approach is sound in the sense that it always detects newly introduced warnings. However, it may over-approximate new warnings in corner cases. For example, if the agent deletes a line with a warning and re-inserts the same line at another location, the warning is flagged as newly introduced.

3.5.3 Running the Tests. In the final step, the change approver runs the project's test suite. If all tests pass, the fix is approved and recorded as plausible. If any tests fail, the fix is rejected. In such cases, the change approver extracts the failure reports from the test execution output and cleans them by removing all stack-trace parts outside the target project. The cleaned reports are then provided as feedback to the agent. Using this information, the agent can attempt to refine its fix to avoid regressions, or alternatively adjust failing test cases if they reflect incorrect expectations.

4 Evaluation

We evaluate CodeCureAgent by addressing the following research questions:

RQ1 What is the effectiveness of CodeCureAgent at fixing static analysis warnings?
RQ2 How efficient is CodeCureAgent with regard to execution time and monetary cost?
RQ3 How does CodeCureAgent compare to state-of-the-art approaches?
RQ4 How do different components of the approach contribute to its effectiveness?

4.1 Evaluation Setup

4.1.1 Dataset. We evaluate CodeCureAgent on a dataset of SonarQube warnings that we create by running the analysis on 132 open-source Java projects used in the Sorald dataset [16]. The Sorald dataset consists of 161 projects, from which we exclude four projects to avoid bias, as these were randomly selected to test and improve our approach during development. In addition, we filter out 25 other projects that cannot be successfully built with Maven or have failing tests. For the remaining 132 Java projects, we fix the projects to the same commits used by Sorald. We use the default SonarWay profile for analysis, which contains 479 rules. By running the analysis, we obtain a total of 95,083 warnings covering 291 distinct rules. We aim to evaluate CodeCureAgent on a representative sample of these warnings, covering as many distinct rules as possible, to make sure

that CodeCureAgent can address arbitrary rules, while also reflecting the distribution of warnings in real-world projects. To achieve this, we first randomly sample one warning for each of the 291 distinct rules, ensuring that we cover a wide variety of rules and violation contexts. This results in an initial set of 291 warnings. To further ensure that our evaluation reflects the real-world distribution of warnings, we then randomly sample an additional 709 warnings. By combining these two sampling strategies, we obtain a dataset of 1,000 warnings spanning 106 projects that we use to evaluate CodeCureAgent.

The static analysis classifies rules into four categories: code smell, bug, security hotspot, and vulnerability.[1] In our dataset, code smells dominate, accounting for 87.7% of all warnings. The remaining warnings include 9.2% bugs, 1.9% security hotspots, and 1.2% vulnerabilities. This is close to the distribution in the full set of 95,083 warnings (95.4% code smells, 3.0% bugs, 1.5% security hotspots, and 0.1% vulnerabilities). The security categories (i.e., security hotspots and vulnerabilities) include rules such as "Delivering code in production with debug features activated is security-sensitive", which can lead to information leakage, and "Server hostnames should be verified during SSL/TLS connections", which, when violated, can make the application susceptible to identity spoofing. A full list of covered security-related rules, can be found in the replication package.

4.1.2 LLM and Hardware. We implement CodeCureAgent on top of the AutoGPT framework and RepairAgent [6]. As the LLM, we use GPT-4.1 mini (version 2025-04-14) from OpenAI. At the time of writing, the cost of the model is $0.4 per million input tokens and $1.6 per million output tokens. The maximum number of cycles is set to 20 and 40 for the classification and repair sub-agents, respectively. We run all experiments in a Docker container on a virtual machine with an Intel Xeon CPU at 2.1 GHz configured with 16 virtual CPUs and 32 GB of RAM.

4.1.3 Baselines. We compare CodeCureAgent against three state-of-the-art techniques.

Sorald [16] is a rule-based technique that implements fixers for 30 analysis rules. When a warning matches a supported rule, Sorald applies the corresponding hand-crafted transformation template. This design makes Sorald straightforward and fast on its covered rules, but coverage is inherently limited, and some rules are only partially supported.

iSMELL [53], an approach focusing on three specific code smell rules, consists of two phases: (1) A detection phase that uses a trained mixture of experts model to select the most accurate static analysis tool for a given code context and code smell rule, to then use this analysis tool to check for the code smell. (2) A refactoring phase that, based on a detected code smell and an example of how the code smell rule can be fixed, queries an LLM to generate a fix. The first phase is orthogonal to our work, and hence, we only compare against the second phase of iSMELL. We use the publicly available reproduction repository[2], which supports modifying a single file at a time. Because the refactoring prompts are hard-coded for the three specific code smells targeted by iSMELL, we generalize them to address any SonarQube rule by prompting the LLM with the rule documentation and the full-file warning context. If the rule documentation does not include an example, we use the LLM to generate one.

Finally, *CORE* [49] is a recent LLM-based technique that can target arbitrary warnings, provided the corresponding rules and their documentation are supplied in a metadata file. For each warning, CORE queries an LLM to propose multiple alternative patches that are ranked using an LLM. Their evaluation counts the warning as "repaired" if at least one candidate patch removes the warning, unlike our change approver, which executes three checks (build, warning removal, and testing).

[1]See https://docs.sonarsource.com/sonarqube-community-build/quality-standards-administration/managing-rules/rules
[2]See https://github.com/iSMELL2024/iSMELL/tree/0dde27c2264354df80cebe379dda9da8f393d6aa

Table 3. Results of CodeCureAgent on the classification and repair task. "Overall" reports results on all 1,000 warnings; "Manually verified" reports on the 291 inspected warnings.

Category	Overall (/1,000)		Manually verified (/291)		
	Agent Classification	Plausible Fix	Correct Classification	Plausible Fix	Correct Fix
True Positive	696	665 (95.6%)	186/191 (97.4%)	178 (93.2%)	170 (89.0%)
False Positive	304	303 (99.7%)	81/100 (81.0%)	81 (81.0%)	81 (81.0%)
All warnings	**1,000**	**968 (96.8%)**	**267 (91.8%)**	**259 (89.0%)**	**251 (86.3%)**

A key design characteristic of CORE is that it can modify a single source file only. CORE was evaluated on a Python dataset with warnings raised by 52 CodeQL static analysis checks and a subset of Sorald's Java dataset with SonarQube, which is limited to 10 different SonarQube rules. We compare against CORE and the other two baselines on our own Java dataset (Section 4.1.1), derived from the set of projects used in Sorald, which covers 291 distinct SonarQube rules. Comparing against CORE on the Python dataset is beyond the scope of this work, as our implementation of CodeCureAgent is specific to Java. We discuss the necessary adaptations to support other languages and static analyzers in Section 5.

For a fair comparison, we run iSMELL and CORE with the same LLM (GPT-4.1 mini) as CodeCureAgent.

4.1.4 Manual Inspection. To evaluate both the correctness of the classification and the proposed fixes, we manually inspect the results for the 291 warnings sampled from all violated rules (Section 4.1.1). Due to the high annotation effort, the warnings and fixes are annotated by a single author of the paper, but unclear cases are discussed between the authors. During annotation, we follow a structured process: For the classification, we examine the warning, its context, and the sub-agent's decision-making process to judge whether the reasoning is sound and consistent with the code and the documentation of the static analysis. To inspect the fixes, we review all modified lines to assess whether TP warnings are resolved in accordance with rule documentation or, in the case of FPs, whether the fix correctly suppresses the warning. We also ensure the revised code remains semantically equivalent to the original, unless intentional differences are justified by addressing an underlying bug. A more detailed description of the annotation process and guidelines is provided in the replication package.

4.2 RQ1: Effectiveness

Table 3 summarizes the results: Out of 1,000 warnings, the agent classifies 696 as TP and 304 as FP, and produces plausible fixes for 968 warnings (96.8%). The plausible-fix rate within each category is similarly high: 665/696 (95.6%) for TPs and 303/304 (99.7%) for FPs. This result indicates that once a warning is labeled, the repair sub-agent is highly effective at addressing both TPs and FPs.

Manual inspection of a representative subset of 291 warnings provides a more precise assessment of the agent's performance by checking both the correctness of the classification and the proposed fixes. We find that the agent correctly classifies 267 of the 291 cases (91.8%). In the repair step, CodeCureAgent attempts fixes for all warnings, but in our manual validation we only judge fixes for correctly classified warnings. Specifically, CodeCureAgent proposes plausible fixes for 259 warnings (89.0%), of which 251 are confirmed as correct, yielding an overall accuracy of 86.3%. Most errors arise in the classification phase (8.2%), rather than in the subsequent repair, which suggests that further improving the classification could positively influence the overall repair success.

Changes to target file `MetricsStream.java`

```
void printPerformanceMetrics() {
-    printf( "%s %d\n", getCountName(), scrapeCount);
-    printf(Locale.US, "%s %.2f\n", getDurationName(), toSeconds(getElapsedTime()));
-    printf(Locale.US, "%s %.2f\n", getCpuUsageName(), toSeconds(getCpuUsed()));
+    printf( "%s %d%n", getCountName(), scrapeCount);
+    printf(Locale.US, "%s %.2f%n", getDurationName(), toSeconds(getElapsedTime()));
+    printf(Locale.US, "%s %.2f%n", getCpuUsageName(), toSeconds(getCpuUsed()));
}
```

Changes to referencing test file `MetricsStreamTest.java`

```
private List<String> getPrintedMetricValues() {
-    return Arrays.stream(getPrintedMetrics().split("\n")).map(l -> l.split(" ")[1]).
     collect(Collectors.toList());
+    return Arrays.stream(getPrintedMetrics().split("\\r?\\n")).map(l -> l.split(" ")
     [1]).collect(Collectors.toList());
}
```

Listing 5. Example of CodeCureAgent fix that successfully resolves the true positive warning of rule S3457 in project weblogic-monitoring-exporter.

```
 public void parse(Message message, String line) throws ParseException
 {
    if( !line.isEmpty() ) {
-       message.setDate(date_format.parse(line));
+       try {
+           ZonedDateTime zdt = ZonedDateTime.parse(line, date_format);
+           message.setDate(Date.from(zdt.toInstant()));
+       } catch (DateTimeParseException e) {
+           throw new ParseException(e.getMessage(), 0);
+       }
    }
+}
 }  // <=== !!Duplicate closing bracket not deleted in the fix
```

Listing 6. Example of CodeCureAgent failed fix attempt because of duplicated bracket (MsgViewer).

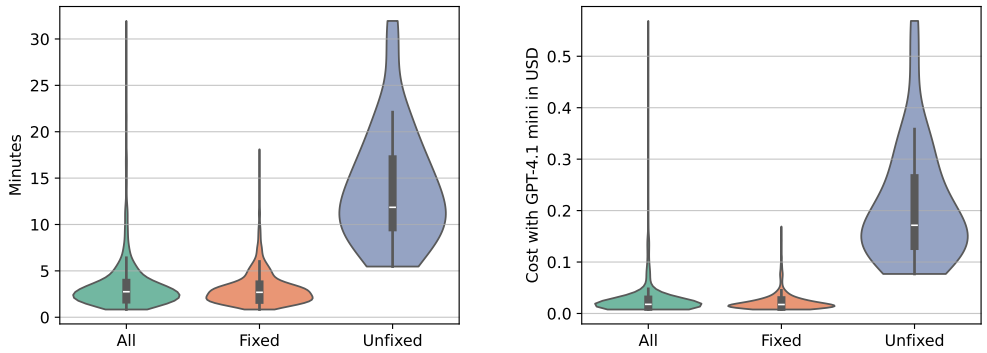
Breaking down the manual results by warning class further highlights this pattern. Among the TPs, the agent classifies 97.4% correctly, generates plausible fixes for 93.2%, and correctly fixes 89.0%. For FPs, the agent classifies 81.0% correctly, with the same percentage for plausible fixes and correct fixes. Thus, the main source of failure is false negatives at the classification stage: CodeCureAgent tends toward missed repairs rather than unwanted changes, since suppression does not alter program behavior beyond silencing the warning.

Listing 5 shows an example of a warning that CodeCureAgent successfully fixes. The warning is raised by rule `S3457`: "Printf-style format strings should be used correctly," which points out that `%n` should be used in place of `\n` to produce a platform-specific line separator. CodeCureAgent correctly classifies this warning as a TP and then replaces three usages of `\n` with `%n` within statements in `MetricsStream.java`. However, after making this change, a test that simulates execution on Windows fails, as it mistakenly uses only `\n` to split the output line by line, instead of the expected `\r\n` on Windows. CodeCureAgent inspects the failing test, identifies the issue, and creates a second fix that additionally modifies the test so that it splits the output by arbitrary line separators. In this way, CodeCureAgent not only resolves the warning but also corrects the faulty test.

An example in which the agent fails to identify a correct fix is shown in Listing 6. The warning to address is `S2885`: "Non-thread-safe fields should not be static". To fix the warning, the agent replaces the static usage of the non-thread-safe `SimpleDateFormat` with the thread-safe `DateTimeFormatter`,

Table 4. Complexity of fixes and plausible-fix rate for 1,000 warnings.

Fix Complexity	All	TP	FP	Plausible
Single-Line	52.0%	33.2%	95.1%	99.4%
Multi-Line	44.4%	61.6%	4.9%	95.5%
Multi-File	3.6%	5.2%	0.0%	75.0%



(a) Time (b) Monetary Cost

Fig. 2. CodeCureAgent time and monetary cost distribution between fixed and unfixed warnings.

and CodeCureAgent attempts to adapt the local call site accordingly. It inserts the full body of the changed method, including an extra closing bracket, thereby duplicating it and breaking the syntactic structure of the class. From the failure feedback provided by the change approver, CodeCureAgent identifies the issue but still fails to correct it within the allotted cycle budget. Future work on expressing code changes without breaking the syntactic structure of the code may help to avoid such issues.

4.2.1 Complexity of Generated Fixes. To better understand the nature of the fixes generated by CodeCureAgent, Table 4 shows the distribution of fixes based on their complexity. Out of 1,000 fixes, 520 affect only a single line, 444 are multi-line fixes, and 36 are multi-file fixes. For warnings classified as FPs, almost all fixes are single-line (95.1%). This is expected, as FPs can usually be suppressed by adding a `NOSONAR` comment to the warning line. In rare cases, however, the suppression spans multiple lines. This can happen, for example, when the reported statement itself is split over several lines and the suppression comment must be added in a way that covers the entire construct. As shown in the last column of the table, less complex fixes are more likely to be plausible: 99.4% of all single-line fixes are plausible, compared to 95.5% for multi-line fixes and an even lower 75.0% for multi-file fixes. This observation shows that while CodeCureAgent is capable of generating complex fixes, the likelihood of success decreases as complexity increases.

4.2.2 Types of Fixed Warnings. When breaking down the results by rule-category (see Section 4.1.1), we find that CodeCureAgent performs robustly across all categories, generating plausible fixes for 849 code smells (96.8%), 90 bugs (97.8%), 17 security hotspots (89.5%), and all 12 vulnerabilities (100.0%).

Table 5. Unified comparison with baselines. "Overall (1,000)" reports results relative to the full dataset. "Sorald-62" are those 62 warnings supported by Sorald. "Sorald-Manual-21" shows the end-to-end correctness on the 21 Sorald-supported warnings that we manually inspected.

	Overall (1,000)		Sorald-62		Sorald-Manual-21
Approach	**Created Fix**	**Plausible Fix**	**Created Fix**	**Plausible Fix**	**Correct Fix**
Sorald	55 (5.5%)	43 (4.3%)	55 (88.7%)	43 (69.4%)	16 (76.2%)
iSMELL	1,000 (100%)	628 (62.8%)	62 (100%)	48 (77.4%)	13 (61.9%)
CORE	926 (92.6%)	676 (67.6%)	57 (91.9%)	47 (75.8%)	14 (66.7%)
CodeCureAgent	1,000 (100%)	968 (96.8%)	62 (100%)	62 (100%)	20 (95.2%)

4.3 RQ2: Efficiency and Costs

4.3.1 Time. Figure 2a shows the distribution of time taken by CodeCureAgent for all warnings, fixed, and unfixed ones. On average, CodeCureAgent takes 4.4 minutes (median=2.8) to go through a warning. For unfixed warnings, the mean is higher at 14.0 minutes. This is expected, as warnings that the agent fails to fix consume the full repair budget of 40 cycles, compared to an average of only 8 repair cycles for fixed warnings. A large fraction of CodeCureAgent's execution time is spent outside of running the main agent loop. On average, 41.1% of the time is attributed to building and testing the target project, and to running the static analysis. In other words, CodeCureAgent's execution time largely depends on the complexity of the project, where projects with complex setups or computationally expensive test suites will need more time to complete. Another 39.2% of time is spent querying the LLM. Using faster models could therefore significantly improve the total execution time.

4.3.2 Token Consumption. The LLM usage of CodeCureAgent incurs a mean token consumption of 139K tokens per warning. Only 3.0% of these tokens are output tokens (which are usually the more expensive tokens). Of the input tokens, 78.8% are cached input tokens, since large parts of the prompts created by CodeCureAgent are unchanged from previous cycles. At the time of writing, cached input tokens are four times less expensive than uncached input tokens. This translates to a mean monetary cost for querying the LLM of 2.9 cents (USD) per warning. Figure 2b compares the monetary cost between fixed and unfixed warnings. Unfixed warnings have a mean cost of 21 cents. This increased cost aligns with the previous findings that unfixed warnings lead to longer execution time and more agent cycles.

4.4 RQ3: Comparison to Baselines

4.4.1 Comparing Effectiveness. Table 5 summarizes the results of comparing CodeCureAgent against Sorald, iSMELL, and CORE. On the full set of 1,000 warnings, CodeCureAgent produces fixes for all warnings and achieves a plausible-fix rate of 96.8%. In contrast, iSMELL drops from 100% *created* to only 62.8% *plausible* (a 37.2% decline), and CORE drops from 92.6% *created* to 67.6% *plausible* (a 25.0% decline), indicating that many candidate fixes fail the oracle checks (builds, tests, or introduction of new warnings). Sorald contributes fixes for only 5.5% of the 1,000 warnings overall, reflecting its limited coverage of the static analysis rules, with only 4.3% plausible fixes after validation.

To investigate the performance on the repair task in isolation of classification, we compare the plausible-fix rates on the subset of warnings that CodeCureAgent classifies as TPs, which are the warnings that require code changes beyond simple suppression. CodeCureAgent achieves a plausible-fix rate of 95.6% on TPs, while Sorald, iSMELL, and CORE have 4.6%, 67.4%, and 75.6%,

respectively. This shows that even when focusing only on the TPs, CodeCureAgent still outperforms the baselines by a significant margin.

The *Sorald-62* part of Table 5 provides a comparison on all 62/1,000 warnings that correspond to rules supported by Sorald. CodeCureAgent attains 100% for both *created* and *plausible* fixes, whereas Sorald reaches 88.7% created and 69.4% plausible fixes. That is, even for the Sorald-supported warnings, CodeCureAgent shows a 30.6% advantage in plausible fixes.

The right-most part of the table shows the results of manually inspecting all 21 warnings from the Sorald-62 set that are part of the manually verified subset of 291 warnings (Section 4.1.1). CodeCureAgent correctly repairs 20/21 of these warnings (95.2%), compared to 16/21 (76.2%) for Sorald, 13/21 (61.9%) for iSMELL, and 14/21 (66.7%) for CORE. The single miss for CodeCureAgent stems from a misclassification into the FP category, which is again consistent with the earlier observation that mistakes by CodeCureAgent predominantly arise during classification.

These results align with the design of the baselines discussed earlier. iSMELL and CORE operate on a single file, which prevents them from addressing multi-file changes. Furthermore, as all baselines do not check for FPs (iSMELL proposes an intelligent selection of analysis tools, but this can only reduce FPs), they are prone to attempt incorrect fixes. For example, *Sorald-62* contains six FPs where attempting to fix the warning according to the rule would break the code, highlighting the importance of reliable classification once again. Together with the quantitative evidence in Table 5, these factors explain why CodeCureAgent achieves both broader applicability and higher reliability across 291 types of warnings.

4.4.2 Comparing Efficiency. We compare CodeCureAgent's efficiency with that of Sorald, iSMELL, and CORE, which we run on the same experimental setup as our approach. Sorald repairs warnings with a mean execution time of 0.5 minutes per warning. This lower execution time is expected, as Sorald is a template-based approach that applies abstract syntax tree transformations, rather than prompting an LLM multiple times. Sorald is even more time-efficient when running on multiple warnings from the same rule at once. iSMELL has a mean execution time of 1.1 minutes and costs 0.9 cents per warning, which is substantially lower than CodeCureAgent's time and cost (4.4 minutes and 2.9 cents). This is because iSMELL prompts the LLM only once per warning to generate a fix, without any further validation steps. CORE's mean execution time is 2.3 minutes per warning, about half that of our approach. At the same time, CORE incurs approximately 4 cents per warning in LLM costs, which is higher than CodeCureAgent's 2.9 cents. These results are due to CORE prompting an LLM multiple times per run with outputs that contain large sections of code, averaging 255 lines of code per patch. In contrast, our approach requires only 73 lines on average for specifying edits, of which only 32 lines indicate changed code lines. The reason why CodeCureAgent takes additional time is that it runs multiple validation steps (build, static analysis, and tests) for approving code changes, which we find beneficial for ensuring high-quality fixes (Section 4.4).

4.5 RQ4: Understanding Different Components of CodeCureAgent

4.5.1 Importance of Checks Done for Change Approval. Table 6 shows three variants of the change approver based on the activated checks. Removing a component means the reduced pipeline no longer validates that property before accepting a candidate patch. A "false accept" is a patch accepted by the reduced checks but rejected by the *full* change approver because it would have failed in build, tests, or in the static analysis check.

As checks are removed, plausibility drops and incorrect fixes slip through. Omitting only tests has a small effect (961 plausible; 9 false accepts), whereas omitting the static analysis check is far more damaging (861 plausible; 123 false accepts). With no checks at all, plausibility falls to

Table 6. Ablation of the change approver on 1,000 warnings. "Checks enforced" indicates which validations are applied: *Build* (project compiles), *No Warning* (target removed; no new static analysis warnings), and *Tests* (existing suite passes). *Plausible fixes* count patches that would also be accepted by the full change approver; *False accepts* are fixes that slip through the ablated variant but would be rejected when all checks run.

	Checks enforced			Plausible fix	Incorrect fix
Variant	**Build**	**No Warning**	**Tests**	/1,000 (%)	**False accept**
Full Change Approval	✓	✓	✓	968 (96.8%)	–
w/o Tests	✓	✓	✗	961 (96.1%)	9
w/o Static Analysis & Tests	✓	✗	✗	861 (86.1%)	123
No Change Approval	✗	✗	✗	751 (75.1%)	249

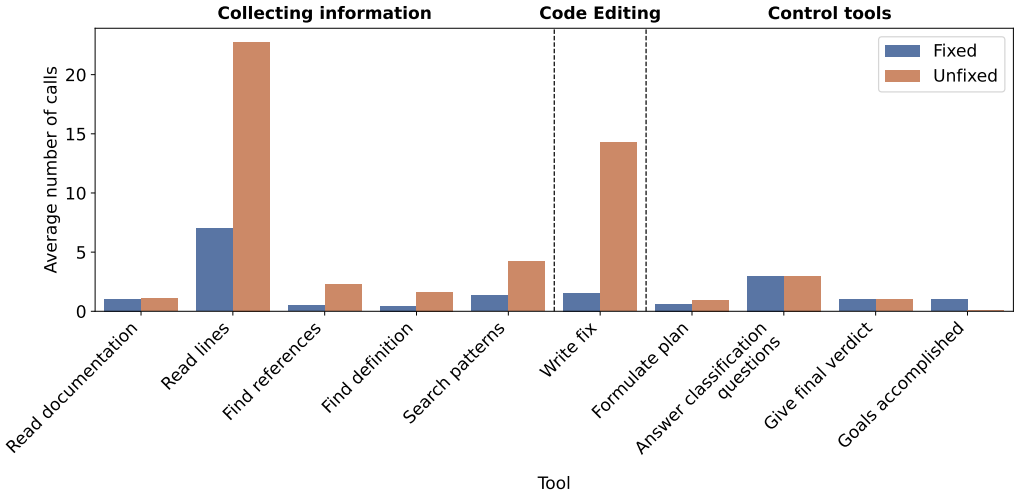


Fig. 3. Absolute number of tool calls, comparing between fixed and unfixed warnings.

751 (75.1%), and 249 incorrect fixes would be admitted, which highlights that the checks and the feedback they provide enable the agent to improve 217 otherwise implausible fixes.

Notably, even in the "No Change Approval" setting, our approach still exceeds the best baseline's plausible rate on the full dataset: 75.1% vs 67.6% for CORE as shown in Table 5, *despite* CORE being allowed to generate up to 10 candidate patches per warning and counting the fix as "plausible" if *at least one* candidate passes. In our evaluation, CodeCureAgent stops at the *first* plausible patch without generating multiple alternatives, yet CodeCureAgent demonstrates better plausible fix generation mainly due to dynamic and incremental context collection, focusing on a single warning at a time as opposed to CORE, and benefiting from the classification phase to avoid suggesting code changes for warnings that should simply be suppressed.

4.5.2 Tools Usage. Figure 3 presents the average number of tool calls for successful and failing instances. CodeCureAgent invokes all the provided tools at varying rates. Among the information-gathering tools, *Read lines* is used most frequently. The tools *Find references* and *Find definition* are used less often, as they are only needed for certain rules where references or definitions may need to be changed or may influence how the warning is addressed.

```
public static int compareNewerIsGreater(RecognisedVersion version1, RecognisedVersion
    version2) {
    return compareNewerIsLower(version2, version1); + //NOSONAR Intentional param inversion
}
```

Listing 7. Example of CodeCureAgent suppression of false positive warning of rule S2234 in project Amidst.

The *Answer classification questions* tool is called three times and the *Give final verdict* tool once on average. This is expected, as there are three questions to answer and one final verdict to give in each run of the classification sub-agent.

The *Write fix* tool is called 1.52 times for fixed warnings and 14.25 times for unfixed warnings. This tool is always needed to propose fixes and will be called multiple times if one or more fix attempts are rejected by the change approver. The average number of calls to the *Write fix* tool therefore corresponds to the number of fixes created per warning.

4.5.3 Causes of success. The following two examples illustrate how CodeCureAgent operates in practice and how its components contribute to successfully resolving warnings.

The first example is a security warning by rule S5042: "Expanding archive files without controlling resource consumption is security-sensitive" in OpenTripPlanner. CodeCureAgent identifies it as a TP, as unchecked expansion can allow for zip bomb attacks. The repair sub-agent then tries to create a fix by adding thresholds for the archive file size and compression ratio, but multiple attempted fixes introduce syntax errors. The change approver's feedback guides the agent towards improved fixes until the build check passes. However, the change approver reports a newly introduced warning, as the fix duplicates a string literal multiple times. The agent reiterates and defines a constant replacing the duplicated literal. After passing this change approver step, two tests fail, as they hit the newly added compression ratio threshold. Using the test failure information, the agent increases the threshold accordingly and thereby creates a fix that is accepted by the change approver. This example illustrates how CodeCureAgent can address challenging warnings by using the change approver checks and feedback to iteratively improve fixes.

In the second example CodeCureAgent suppresses a FP, shown in Listing 7. The warning is about arguments `version1` and `version2` being passed in reverse order to the `compareNewerIsLower` method. To classify the warning, the classification sub-agent first uses tools to retrieve the rule documentation and the code context around the warning. At this point, the agent already suspects that the inversion is intentional, as the method implements the inverted comparison result. To confirm this hypothesis, the agent additionally retrieves all references to `compareNewerIsGreater` and `compareNewerIsLower`. Based on this context, the agent is certain that the inversion is intentional, answers the classification questions and gives the final verdict that the warning is a FP, which is then suppressed by the repair sub-agent. This example demonstrates how the agent can collect various types of context on demand to make informed decisions and avoid incorrect fixes for FPs.

4.5.4 Causes of Failures.

Causes of wrong classification. In the 291 inspected warnings, we find 24 misclassified warnings, which represent 8.2% of all inspected warnings; 19 wrongly classified as FP. Incorrect classifications as TP can lead to the fix attempt breaking intended functionality or breaking the build entirely. On the other hand, incorrect classifications as FP lead to the suppression of an actually valid warning. Upon examination, we find that the failed classifications can be attributed to four main reasons:

(1) LLM hallucination (11/24): The LLM ignores given context and outputs a decision that is contradictory to indications available in the context.

(2) Insufficient context (11/24): Sometimes the static analysis rule is not well documented or imprecise (9/24), and other times the collected context around the warning is shallow (2/24) making the LLM decision almost arbitrary.
(3) Other reasons (2/24): Edge cases, where the developer follows a convention that the LLM does not agree with.

Causes of inability to repair. The repair sub-agent fails to (plausibly) fix 32 warnings. Furthermore, 8 inspected plausible fixes are incorrect. The failure of repair can be attributed to three main reasons: (i) Inability to fix incorrect modifications that are breaking code in 26/32 instances (ii) addressing multiple warnings at once in 4/32 instances, despite being instructed to focus on one at a time, and (iii) no support for creating, renaming, or deleting files, meaning a limitation within the available tools in 2/32 cases.

5 Threats to Validity and Limitations

We focus on one static analysis (SonarQube) and one programming language (Java). While both are widely used and highly relevant in practice, the generalizability of our findings to other static analyzers and languages remains to be validated. To adapt CodeCureAgent to another static analyzer, the new analyzer needs to be integrated by making it invokable with a single command, mapping its output to the expected JSON format, and updating the *Read Documentation* tool. Adapting CodeCureAgent to another programming language requires using a different parser and language server, and to adapt the build and test commands to the new language. Moreover, code examples in the LLM prompts would need to be updated. However, the overall approach is not specific to Java and SonarQube, and we expect it to be applicable to other settings.

Our dataset, while diverse, may not capture all types of warnings or project structures, potentially limiting the generalizability of our results. However, with 291 distinct rules covered, our dataset is an order of magnitude broader than those evaluated by prior work [16, 49], which cover 10 and 52 rules, respectively.

The manual inspection process, despite efforts to ensure consistency, may introduce bias in classification and fix correctness assessments, especially as there is only a single annotator who is one of the authors. To reduce this risk, we follow a systematic annotation process and discuss unclear cases (Section 4.1.4).

Additionally, the reliance on GPT-4.1 mini as the LLM may influence results, and different models could yield varying performance. As stronger models become available and economically viable, we expect CodeCureAgent's performance to improve further. For a fair comparison with the LLM-based baseline [49], we use the same LLM as for CodeCureAgent. Another threat is data leakage. While the LLM might have been pre-exposed to the source code of the projects in our dataset, it has neither seen the exact warnings in the code nor any respective fixes, as the projects generally do not use SonarQube and do not address warnings in dedicated commits. Therefore, the LLM cannot directly recall a fix. Furthermore, the baselines are all evaluated with the same LLM, so any potential data leakage would affect all approaches equally. Finally, our evaluation primarily focuses on the technical effectiveness, without assessing its impact on developer workflows or acceptance in real-world settings. Studying how developers interact with and perceive the technique is an important future work.

6 Related Work

Static Analysis Tools. Range from general-purpose analyzers to domain-specific checkers including SonarQube [15], CodeQL with declarative, multi-language analyses [12, 57], and Infer. Some tools are more specific, such as NullAway for Java nullness [3, 26], test smell detectors [50],

and tensor-shape analysis [29]. Work on learning or synthesizing analysis rules aims to reduce the manual effort of rule creation and maintenance [14, 17]. Complementary efforts build specifications (e.g., taint specs for JS libraries) to strengthen downstream analyses [46]. Empirical studies further tackle aspects of usage in practice and limitations [19, 59].

Detecting and Reducing False Positives. A first task in mitigation is filtering false alarms. Machine learning techniques classify warnings into TPs and FPs [52, 59], sometimes validating with targeted tests [25]. Beyond classification, several works reduce analysis imprecision at the source: pruning false call-graph edges [42, 48], neural assistance to refine static analysis [34, 58]. Recent LLM-assisted approaches treat a warning as a structured query containing code and project context [34, 52].

Automated Repair of Warnings. Data-driven repair learns edit patterns from historical fixes and applies them [2, 5, 35]. Rule-based repair (e.g., Sonar quick-fixes and Sorald) shows strong precision but can struggle to generalize beyond encoded idioms [15]. StaticFixer augments static warnings with code transformations [21], while synthesis/constraint-driven approaches use analysis goals to guide patching [37]. For type-related violations, specialized techniques automatically fix type errors in Python and functional languages [11, 41, 44], and learned type hints can trigger or resolve static checker feedback [1]. Beyond single-rule scripts, LLM-centric pipelines combine transformation-by-example with static/dynamic checks to produce robust edits [13].

General Automated Repair Approaches. A broad line of work in automated program repair has explored how to generate patches for general software defects [32]. Early systems relied on generate and validate strategies such as GenProg [31] or constraint solving, as in SemFix [27], to search for program variants that satisfy the test suite. Template and pattern-based approaches later emerged to capture common fix idioms with higher precision [36, 38], and several such techniques have been deployed in industrial settings [2, 40]. The advent of learning-based repair shifted attention toward models trained directly on large corpora of bug fixes [8, 39, 47, 55, 56] and also type and context-aware repair frameworks [22, 45]. More recently, general-purpose large language models have been adapted for repair [54], and researchers have begun to orchestrate them with agentic prompting pipelines that couple generation with validation and feedback [6, 9]. A particularly related work is RepairAgent [6], which also uses an LLM agent with custom designed tools. However, RepairAgent targets bug repair with a given bug-revealing test case, while CodeCureAgent focuses on fixing static analysis warnings. Most importantly, RepairAgent assumes that all bugs are TPs, whereas CodeCureAgent first classifies warnings, which is crucial for avoiding unnecessary or harmful code changes. In addition CodeCureAgent uses specialized tools for collecting context and validating fixes that are distinct from RepairAgent. Lastly, CodeCureAgent is even capable of fixing wrong or inaccurate test cases (e.g., see Listing 5).

7 Conclusion

In this paper, we present CodeCureAgent, an autonomous LLM agent that first classifies static analysis warnings as TPs or FPs, then either repairs or suppresses them. A built-in change approver validates each edit by ensuring the project builds successfully, the original warning is eliminated, no new warnings arise, and all tests pass. Evaluated on 1,000 SonarQube warnings across 106 Java projects, CodeCureAgent generates plausible fixes for 96.8% of warnings, correctly classifies 91.8% of distinct-rule cases, and achieves an overall end-to-end correct fix rate of 86.3%, surpassing prior techniques such as Sorald, iSMELL, and CORE. By addressing multi-line and multi-file fixes, and by filtering FPs, CodeCureAgent significantly reduces the manual effort needed to maintain clean codebases for a cost of only a few cents and a couple of minutes per warning. Future work could

expand language support, target new static analyzers, and incorporate developer feedback for continual learning and integration of developer intent.

8 Data Availability

The code and data are publicly available at https://github.com/sola-st/CodeCureAgent and archived with a DOI on Zenodo [24].

References

[1] Miltiadis Allamanis, Earl T. Barr, Soline Ducousso, and Zheng Gao. 2020. Typilus: neural type hints. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI*. 91–105. doi:10.1145/3385412.3385997

[2] Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. 2019. Getafix: Learning to fix bugs automatically. *Proc. ACM Program. Lang.* 3, OOPSLA (2019), 159:1–159:27. doi:10.1145/3360585

[3] Subarno Banerjee, Lazaro Clapp, and Manu Sridharan. 2019. NullAway: practical type-based null safety for Java. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019*, Marlon Dumas, Dietmar Pfahl, Sven Apel, and Alessandra Russo (Eds.). ACM, 740–750. doi:10.1145/3338906.3338919

[4] Patrick Bareiß, Beatriz Souza, Marcelo d'Amorim, and Michael Pradel. 2022. Code Generation Tools (Almost) for Free? A Study of Few-Shot, Pre-Trained Language Models on Code. *CoRR* abs/2206.01335 (2022). arXiv:2206.01335 doi:10.48550/arXiv.2206.01335

[5] Rohan Bavishi, Hiroaki Yoshida, and Mukul R. Prasad. 2019. Phoenix: automated data-driven synthesis of repairs for static analysis violations. In *ESEC/FSE*. 613–624. doi:10.1145/3338906.3338952

[6] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *International Conference on Software Engineering (ICSE)*.

[7] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching Large Language Models to Self-Debug. arXiv:2304.05128 [cs.CL]

[8] Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2021. SequenceR: Sequence-to-Sequence Learning for End-to-End Program Repair. *IEEE Trans. Software Eng.* 47, 9 (2021), 1943–1959. doi:10.1109/TSE.2019.2940179

[9] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic Bug Reproduction for Effective Automated Program Repair at Google. *arXiv preprint arXiv:2502.01821* (2025).

[10] Yiu Wai Chow, Luca Di Grazia, and Michael Pradel. 2024. PyTy: Repairing Static Type Errors in Python. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, 1–13. doi:10.1145/3597503.3639184

[11] Yiu Wai Chow, Luca Di Grazia, and Michael Pradel. 2024. PyTy: Repairing Static Type Errors in Python. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 87:1–87:13. doi:10.1145/3597503.3639184

[12] Yiu Wai Chow, Max Schäfer, and Michael Pradel. 2023. Beware of the Unexpected: Bimodal Taint Analysis. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA*, René Just and Gordon Fraser (Eds.). ACM, 211–222. doi:10.1145/3597926.3598050

[13] Malinda Dilhara, Abhiram Bellur, Timofey Bryksin, and Danny Dig. 2024. Unprecedented Code Change Automation: The Fusion of LLMs and Transformation by Example. In *FSE*. https://doi.org/10.48550/arXiv.2402.07138

[14] Sedick David Baker Effendi, Berk Cirisci, Rajdeep Mukherjee, Hoan Nguyen, and Omer Tripp. 2023. A language-agnostic framework for mining static analysis rules from code changes. In *ICSE-SEIP*.

[15] Khashayar Etemadi, Nicolas Harrand, Simon Larsén, Haris Adzemovic, Henry Luong Phu, Ashutosh Verma, Fernanda Madeiral, Douglas Wikström, and Martin Monperrus. 2023. Sorald: Automatic Patch Suggestions for SonarQube Static Analysis Violations. *IEEE Trans. Dependable Secur. Comput.* 20, 4 (2023), 2794–2810. doi:10.1109/TDSC.2022.3167316

[16] Khashayar Etemadi, Nicolas Harrand, Simon Larsén, Haris Adzemovic, Henry Luong Phu, Ashutosh Verma, Fernanda Madeiral, Douglas Wikström, and Martin Monperrus. 2023. Sorald: Automatic Patch Suggestions for SonarQube Static Analysis Violations. *IEEE Transactions on Dependable and Secure Computing* 20, 4 (7 2023), 2794–2810. doi:10.1109/TDSC.2022.3167316

[17] Pranav Garg and Srinivasan Sengamedu. 2022. Example-based Synthesis of Static Analysis Rules. *CoRR* abs/2204.08643 (2022). arXiv:2204.08643 doi:10.48550/arXiv.2204.08643

[18] Alex Gu, Wen-Ding Li, Naman Jain, Theo X. Olausson, Celine Lee, Koushik Sen, and Armando Solar-Lezama. 2024. The Counterfeit Conundrum: Can Code Language Models Grasp the Nuances of Their Incorrect Generations? arXiv:2402.19475 [cs.SE]

[19] Huimin Hu, Yingying Wang, Julia Rubin, and Michael Pradel. 2025. An Empirical Study of Suppressed Static Analysis Warnings. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 290–311.
[20] Nasif Imtiaz, Akond Rahman, Effat Farhana, and Laurie Williams. 2019. Challenges with responding to static analysis tool alerts. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 245–249. doi:10.1109/MSR.2019.00049
[21] Naman Jain, Shubham Gandhi, Atharv Sonwane, Aditya Kanade, Nagarajan Natarajan, Suresh Parthasarathy, Sriram Rajamani, and Rahul Sharma. 2023. StaticFixer: From Static Analysis to Static Repair. arXiv:2307.12465 [cs.SE]
[22] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *ICSE*. 1430–1442. doi:10.1109/ICSE48619.2023.00125
[23] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why don't software developers use static analysis tools to find bugs?. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 672–681. doi:10.1109/ICSE.2013.6606613
[24] Pascal Joos, Islem Bouzenia, and Michael Pradel. 2026. CodeCureAgent: Automatic Classification and Repair of Static Analysis Warnings. Software and data archived in Zenodo. doi:10.5281/zenodo.19552529
[25] Ashwin Kallingal Joshy, Xueyuan Chen, Benjamin Steenhoek, and Wei Le. 2021. Validating Static Warnings via Testing Code Fragments. In *ISSTA*.
[26] Nima Karimipour, Justin Pham, Lazaro Clapp, and Manu Sridharan. 2023. Practical Inference of Nullability Types. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2023, San Francisco, CA, USA, December 3-9, 2023*, Satish Chandra, Kelly Blincoe, and Paolo Tonella (Eds.). ACM, 1395–1406. doi:10.1145/3611643.3616326
[27] Yalin Ke, Kathryn T Stolee, Claire Le Goues, and Yuriy Brun. 2015. Repairing programs with semantic code search (t). In *ASE*. IEEE, 295–306.
[28] Anant Kharkar, Roshanak Zilouchian Moghaddam, Matthew Jin, Xiaoyu Liu, Xin Shi, Colin B. Clement, and Neel Sundaresan. 2022. Learning to Reduce False Positives in Analytic Bug Detectors. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE*. 1307–1316. doi:10.1145/3510003.3510153
[29] Sifis Lagouvardos, Julian Dolby, Neville Grech, Anastasios Antoniadis, and Yannis Smaragdakis. 2020. Static Analysis of Shape in TensorFlow Programs. In *34th European Conference on Object-Oriented Programming, ECOOP*, Vol. 166. 15:1–15:29. doi:10.4230/LIPIcs.ECOOP.2020.15
[30] Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W. O'Hearn. 2022. Finding real bugs in big programs with incorrectness logic. *Proc. ACM Program. Lang.* 6, OOPSLA (2022), 1–27. doi:10.1145/3527325
[31] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Trans. Software Eng.* 38, 1 (2012), 54–72.
[32] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. 2019. Automated program repair. *Commun. ACM* 62, 12 (2019), 56–65. doi:10.1145/3318162
[33] Haonan Li, Yu Hao, Yizhuo Zhai, and Zhiyun Qian. 2023. The Hitchhiker's Guide to Program Analysis: A Journey with Large Language Models. arXiv:2308.00245 [cs.SE]
[34] Ziyang Li, Saikat Dutta, and Mayur Naik. 2024. LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. arXiv:2405.17238 [cs.CR]
[35] Kui Liu, Dongsun Kim, Tegawendé F Bissyandé, Shin Yoo, and Yves Le Traon. 2018. Mining fix patterns for findbugs violations. *IEEE Transactions on Software Engineering* (2018).
[36] Kui Liu, Anil Koyuncu, Dongsun Kim, and Tegawendé F. Bissyandé. 2019. TBar: revisiting template-based automated program repair. In *ISSTA*. ACM, 31–42. doi:10.1145/3293882.3330577
[37] Yu Liu, Sergey Mechtaev, Pavle Subotić, and Abhik Roychoudhury. 2023. Program Repair Guided by Datalog-Defined Static Analysis. In *ESEC/FSE*. 1216–1228.
[38] Fan Long and Martin Rinard. 2016. Automatic patch generation by learning correct code. In *POPL*. 298–312.
[39] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. CoCoNuT: combining context-aware neural translation models using ensemble for program repair. In *ISSTA*. ACM, 101–114. doi:10.1145/3395363.3397369
[40] Alexandru Marginean, Johannes Bader, Satish Chandra, Mark Harman, Yue Jia, Ke Mao, Alexander Mols, and Andrew Scott. 2019. Sapfix: Automated end-to-end repair at scale. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*.
[41] Wonseok Oh and Hakjoo Oh. 2022. PyTER: Effective Program Repair for Python Type Errors. In *ESEC/FSE*.
[42] Michael Reif, Florian Kübler, Michael Eichberg, Dominik Helm, and Mira Mezini. 2019. Judge: identifying, understanding, and evaluating sources of unsoundness in call graphs. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019, Beijing, China, July 15-19, 2019*, Dongmei Zhang and Anders Møller (Eds.). ACM, 251–261. doi:10.1145/3293882.3330555

[43] Nick Rutar, Christian B. Almazan, and Jeffrey S. Foster. 2004. A Comparison of Bug Finding Tools for Java. In *International Symposium on Software Reliability Engineering (ISSRE)*. IEEE Computer Society, 245–256.

[44] Georgios Sakkas, Madeline Endres, Benjamin Cosman, Westley Weimer, and Ranjit Jhala. 2020. Type error feedback via analytic program repair. In *Proceedings of the 41st ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2020, London, UK, June 15-20, 2020*, Alastair F. Donaldson and Emina Torlak (Eds.). ACM, 16–30. doi:10.1145/3385412.3386005

[45] André Silva, Sen Fang, and Martin Monperrus. 2024. RepairLLaMA: Efficient Representations and Fine-Tuned Adapters for Program Repair. arXiv:2312.15698 [cs.SE]

[46] Cristian-Alexandru Staicu, Martin Toldam Torp, Max Schäfer, Anders Møller, and Michael Pradel. 2020. Extracting taint specifications for JavaScript libraries. In *ICSE '20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020*, Gregg Rothermel and Doo-Hwan Bae (Eds.). ACM, 198–209. doi:10.1145/3377811.3380390

[47] Michele Tufano, Jevgenija Pantiuchina, Cody Watson, Gabriele Bavota, and Denys Poshyvanyk. 2019. On learning meaningful code changes via neural machine translation. In *ICSE*. 25–36. https://dl.acm.org/citation.cfm?id=3339509

[48] Akshay Utture, Shuyang Liu, Christian Gram Kalhauge, and Jens Palsberg. 2022. Striking a Balance: Pruning False-Positives from Static Call Graphs. In *ICSE*.

[49] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, and Sriram Rajamani. 2024. CORE: Resolving Code Quality Issues using LLMs. *Proceedings of the ACM on Software Engineering* 1, FSE (7 2024), 789–811. doi:10.1145/3643762

[50] Tongjie Wang, Yaroslav Golubev, Oleg Smirnov, Jiawei Li, Timofey Bryksin, and Iftekhar Ahmed. 2021. PyNose: A Test Smell Detector For Python. In *ASE*.

[51] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2024. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741 [cs.SE] https://arxiv.org/abs/2407.16741

[52] Cheng Wen, Yuandao Cai, Bin Zhang, Jie Su, Zhiwu Xu, Dugang Liu, Shengchao Qin, Zhong Ming, and Cong Tian. 2024. Automatically Inspecting Thousands of Static Bug Warnings with Large Language Model: How Far Are We? *ACM Transactions on Knowledge Discovery from Data* (2024).

[53] Di Wu, Fangwen Mu, Lin Shi, Zhaoqiang Guo, Kui Liu, Weiguang Zhuang, Yuqi Zhong, and Li Zhang. 2024. iSMELL: Assembling LLMs with Expert Toolsets for Code Smell Detection and Refactoring. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering* (Sacramento, CA, USA) *(ASE '24)*. Association for Computing Machinery, New York, NY, USA, 1345–1357. doi:10.1145/3691620.3695508

[54] Chunqiu Steven Xia and Lingming Zhang. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 819–831. doi:10.1145/3650212.3680323

[55] He Ye, Matias Martinez, and Martin Monperrus. 2022. Neural Program Repair with Execution-based Backpropagation. In *ICSE*.

[56] He Ye and Martin Monperrus. 2024. ITER: Iterative Neural Repair for Multi-Location Patches. In *ICSE*.

[57] Dongjun Youn, Sungho Lee, and Sukyoung Ryu. 2023. Declarative static analysis for multilingual programs using CodeQL. *Softw. Pract. Exp.* 53, 7 (2023), 1472–1495. doi:10.1002/spe.3199

[58] Jinman Zhao, Aws Albarghouthi, Vaibhav Rastogi, Somesh Jha, and Damien Octeau. 2018. Neural-augmented static analysis of Android communication. In *Proceedings of the 2018 ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2018, Lake Buena Vista, FL, USA, November 04-09, 2018*. 342–353. doi:10.1145/3236024.3236066

[59] Yunhui Zheng, Saurabh Pujar, Burn L. Lewis, Luca Buratti, Edward A. Epstein, Bo Yang, Jim Laredo, Alessandro Morari, and Zhong Su. 2021. D2A: A Dataset Built for AI-Based Vulnerability Detection Methods Using Differential Analysis. In *43rd IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP) 2021, Madrid, Spain, May 25-28, 2021*. IEEE, 111–120. doi:10.1109/ICSE-SEIP52600.2021.00020